Prácticas de Ampliación de Sistemas Operativos • `watchdir`

Descripción del programa `watchdir`

Escribe un programa llamado `watchdir` que monitorice el contenido de un directorio a intervalos regulares, mostrando un mensaje en un archivo de *log* cada vez que se crea, actualiza o elimina un archivo en el mismo. Por ejemplo:

```
$ mkdir -p wd
$ touch wd/f{0..2}
$ ls wd/
total 0
-rw-rw---- 1 alumno alumno 0 sep 30 12:00 f0
-rw-rw---- 1 alumno alumno 0 sep 30 12:00 f1
-rw-rw---- 1 alumno alumno 0 sep 30 12:00 f2
$ (./watchdir -n 2 -l wd.log wd/ &) ; sleep 1 ; cat wd.log
Creation: f0
Creation: f1
Creation: f2
$ touch wd/f3 ; sleep 2 ; cat wd.log
Creation: f0
Creation: f1
Creation: f2
Creation: f3
$ killall watchdir
```

Ten en cuenta que en el ejemplo anterior, como la frecuencia de monitorización es de 2 segundos, debemos esperar 2 segundos para estar seguros de que el archivo de *log* ha sido actualizado por `watchdir` antes de consultar su contenido.

El programa tiene que aceptar las siguientes opciones:

```
Usage: ./watchdir [-n SECONDS] [-l LOG] [DIR]
    SECONDS Refresh rate in [1..60] seconds [default: 1].
    LOG     Log file [default: '/tmp/watchdir.log'].
    DIR     Directory name [default: '.'].
```

La ayuda se muestra **en inglés y tal cual aparece arriba**: los tests la comprueban carácter a carácter y cada línea de opciones empieza con un tabulador.

Consideraciones, indicaciones y requisitos

- Los argumentos de la línea de órdenes debe procesarse con `getopt()` y no podrá suponerse un orden concreto de dichos argumentos. Además podrá utilizarse más de una vez un argumento, en cuyo caso se tomará la última aparición del mismo como la correcta.

- `SECONDS` indica con qué frecuencia se tienen que ejecutar las tareas de registro del contenido del directorio, comparación del registro actual con el registro anterior y, por último, actualización del archivo de *log* con las diferencias entre ambos registros. Por tanto, `watchdir` tendrá que registrar el contenido del directorio tan pronto como se ejecuta. A partir de entonces, cada vez que pasen los segundos indicados, `watchdir` registrará de nuevo el contenido del directorio. A continuación, se añadirán al archivo de *log* las diferencias entre los dos últimos registros.

- `LOG` es el nombre del archivo de *log*. Cada vez que se ejecuta `watchdir`, se creará el archivo de *log* si no existe. Si éste existe, se descartará su contenido. Las diferencias a incluir en el archivo de *log* tendrán el siguiente formato:

    - `Creation: FILENAME`
    - `UpdateName: OLD_FILENAME -> NEW_FILENAME`
    - `UpdateSize: FILENAME: OLD_SIZE -> NEW_SIZE`
    - `UpdateMtim: FILENAME: OLD_MTIM -> NEW_MTIM`
    - `Deletion: FILENAME`

    El formato de `MTIM`, que es la fecha y hora de la última modificación de la entrada, será `AAAA-MM-DD HH:MM:SS`, por ejemplo, `2026-09-30 13:00:00`.

    Por simplicidad, sólo se registrará un cambio por entrada con el siguiente orden de prioridad: creación, borrado, cambio de nombre, cambio de tamaño, cambio de fecha de última modificación. Por ejemplo, si cambia el tamaño de un archivo, sólo tendría que registrarse el cambio de tamaño pero no el cambio de fecha de última modificación.

- `DIR` es el nombre del archivo que debe monitorizarse a intervalos regulares.

- La ejecución de `watchdir` no termina nunca, es decir, `watchdir` realiza las tareas de registro, comparación y actualización del archivo de *log* a intervalos regulares dentro de un bucle infinito. Para conseguirlo, se debe usar la llamada al sistema `setitimer` para que la **señal** `SIGALRM` se envíe al proceso `watchdir` con la cadencia indicada por `SECONDS`. Forma parte de vuestra tarea consultar la página del manual de la llamada al sistema `setitimer` para aprender a usarla. Un manejador de señales para la señal `SIGALRM` será el encargado de ejecutar las tareas anteriores cada vez que se reciba la señal. Téngase en cuenta que, para evitar que el bucle infinito consuma recursos del sistema innecesariamente, se podría suspender la ejecución del proceso hasta que se reciba la señal.

- Además de procesar la señal `SIGALRM`, `watchdir` tendrá que procesar también la **señal** `SIGUSR1`. Cuando se reciba esta señal, se descartará el contenido del archivo de *log*.

- Para llevar a cabo la **tarea de registro** del contenido del directorio, se empleará la llamada al sistema `scandir`. Forma parte de vuestra tarea consultar la página del manual de la llamada al sistema `scandir` para aprender a usarla. Esta llamada al sistema permite obtener las entradas de un directorio, pudiendo

usar funciones auxiliares para filtrar y ordenar las entradas. Las únicas entradas del directorio que deben excluirse del registro son `.` y `..`

- La estructura `struct dirent` está definida en `dirent.h` de la siguiente forma:

```
struct dirent {
    ino_t          d_ino;       /* Inode number */
    off_t          d_off;       /* Not an offset; see below */
    unsigned short d_reclen;    /* Length of this record */
    unsigned char  d_type;      /* Type of file; not supported
                                   by all filesystem types */
    char           d_name[256]; /* Null-terminated filename */
};
```

- La primera vez que se lea el contenido del directorio, se deberá escribir en el archivo de *log* que los ficheros fueron creados.
- La **tarea de comparación** del registro actual del contenido del directorio con el registro anterior se tendrá que realizar con un algoritmo de complejidad lineal en tiempo, es decir, O(n). Nótese que el uso adecuado de la función auxiliar de ordenación de entradas de `scandir` así lo permite.
- Para simplificar la tarea de actualización, los nombres de las entradas de la secuencia de diferencias no tendrán por qué aparecer en orden alfabético, sino que podrán hacerlo en cualquier orden.
- Durante la **tarea de actualización** se incluirán las diferencias entre los dos últimos registros de las entradas del directorio, conforme al formato arriba indicado, en el archivo de *log*.
- Con objeto de facilitar la implementación de la tarea de actualización del archivo de *log*, a diferencia de la tarea anterior, se permite el uso de las funciones de la biblioteca estándar de C `fopen()` y `fclose()` junto con `fprintf()`. Adicionalmente, se permite el uso de `printf()` y `fprintf()` para mostrar mensajes de ayuda o de error.
- Se prohíbe el uso de funciones de manipulación de cadenas como, por ejemplo, `strcpy()` o `strcat()`, y similares, por no ser seguras al no controlar el número de bytes que copian, por lo que podrían exceder el tamaño de la memoria reservada para el destino. Por tanto, para evitar errores difíciles de depurar debidos a corrupción de memoria, solo se permite el uso de `strncmp()` y `strncpy()` para manipular los nombres de las entradas que devuelve la llamada al sistema `scandir()`.

Ejemplos de ejecución de `watchdir`

```
$ ./watchdir -h
Usage: ./watchdir [-n SECONDS] [-l LOG] [DIR]
    SECONDS Refresh rate in [1..60] seconds [default: 1].
    LOG     Log file [default: '/tmp/watchdir.log'].
    DIR     Directory name [default: '.'].

$ echo $?
0
$ ./watchdir -j
```

```
./watchdir: invalid option -- 'j'
Usage: ./watchdir [-n SECONDS] [-l LOG] [DIR]
    SECONDS Refresh rate in [1..60] seconds [default: 1].
    LOG     Log file [default: '/tmp/watchdir.log'].
    DIR     Directory name [default: '.'].

$ echo $?
1
$ mkdir -p wd
$ touch wd/f{0..2}
$ ls wd/
total 0
-rw-rw---- 1 alumno alumno 0 sep 30 12:00 f0
-rw-rw---- 1 alumno alumno 0 sep 30 12:00 f1
-rw-rw---- 1 alumno alumno 0 sep 30 12:00 f2
$ (./watchdir -n 2 -l wd.log wd/ &) ; sleep 1 ; cat wd.log
Creation: f0
Creation: f1
Creation: f2
$ touch wd/f3 ; sleep 2 ; cat wd.log
Creation: f0
Creation: f1
Creation: f2
Creation: f3
$ mv wd/f0 wd/f4 ; sleep 2 ; cat wd.log
Creation: f0
Creation: f1
Creation: f2
Creation: f3
UpdateName: f0 -> f4
$ echo 1 > wd/f1 ; sleep 2 ; cat wd.log
Creation: f0
Creation: f1
Creation: f2
Creation: f3
UpdateName: f0 -> f4
UpdateSize: f1: 0 -> 2
```

```
$ touch wd/f1 ; sleep 2 ; cat wd.log
Creation: f1
Creation: f2
Creation: f0
Creation: f3
UpdateName: f0 -> f4
UpdateSize: f1: 0 -> 2
UpdateMtim: f1: 2026-09-30 12:00:00 -> 2026-09-30 12:01:00
$ rm -f wd/f4 ; sleep 2 ; cat wd.log
Creation: f1
Creation: f2
Creation: f0
Creation: f3
```

```
UpdateName: f0 -> f4
UpdateSize: f1: 0 -> 2
UpdateMtim: f1: 2026-09-30 12:00:00 -> 2026-09-30 12:01:00
Deletion: f4
$ ps
    PID TTY          TIME CMD
   1234 pts/1    00:00:00 bash
   5678 pts/1    00:00:00 watchdir
   9012 pts/1    00:00:00 ps
$ killall watchdir
$ ps
    PID TTY          TIME CMD
   1234 pts/1    00:00:00 bash
   9012 pts/1    00:00:00 ps
```

Depuración

Para hacer pruebas mientras estáis desarrollando el programa no es imprescindible haber implementado el tratamiento de las señales. Una forma sencilla de conseguir comprobar si estáis detectando bien los cambios en el directorio, es usar una llamada a `sleep(n)` dentro de un bucle, siendo `n` el número de segundos que hay que esperar. Por supuesto, **ESTA SOLUCIÓN NO SERÍA CORRECTA** y por eso la llamada `sleep()` está anulada en la ejecución de los tests. Una vez comprobado que las comparaciones funcionan, podríais incorporarlas en el manejador de la señal `SIGALRM`.

Utilizando esta estrategia es posible depurar el programa con Visual Studio Code modificando la entrada `args` del archivo `launch.json`, tal y como se explicó durante la primera sesión de prácticas.

Entrega y evaluación

- En la entrega se incluirá únicamente el archivo `watchdir.c`.
- El archivo **deberá compilar** conforme a los archivos `tasks.json` o `Makefile` incluidos en `ASO-Semana1.tar.xz`, es decir, con `gcc -ggdb3 -Wall -Werror -Wno-unused -std=c11 watchdir.c -o watchdir`.
- La primera línea de `watchdir.c` deberá ser la directiva para compilar conforme al estándar POSIX: `#define _POSIX_C_SOURCE 200809L`.
- El fichero `watchdir.c` **deberá pasar todos los tests** proporcionados en `watchdir.json` para la utilidad de validación `asosystest.py`.

Para aprobar la práctica es imprescindible que el programa pase las pruebas de ejecución, sin que esto signifique que no se puedan detectar otros errores con pruebas adicionales que el profesor haga durante la evaluación.

Se tendrán en cuenta la correcta indentación del código, su estructuración, el manejo de los posibles errores de las llamadas al sistema o funciones de biblioteca, el uso de constantes en lugar de *números mágicos*, así como la correcta gestión de la memoria dinámica. Otro punto importante, y por eso lo volvemos a recordar, es que se penalizará severamente el no usar buffers para las lecturas/escrituras, es decir, leer o escribir byte a byte, pudiendo llegar a suspender la práctica, así como no contemplar lecturas o escrituras parciales si así se

pide en el enunciado. Adicionalmente, las ineficiencias que se detecten en el código también podrán ser objeto de penalizaciones.

Referencias

- Linux Library Functions Manual: scandir
- Linux POSIX Programmer's Manual: getitimer and setitimer